A Hybrid Web 2.5 Architecture for Decentralized Vehicle Rental Management Using NFT-Based Vehicle Registry

1st Omansh Sharma
SCOPE
VIT
Vellore, India
omaansh@gmail.com

***Abstract*—The proliferation of blockchain technology has enabled the development of decentralized applications (dApps) that promise transparency, security, and disintermediation in peer-to-peer marketplaces. However, purely on-chain architectures suffer from performance bottlenecks, high transaction costs, and poor user experience, limiting mainstream adoption. This paper presents dVRM (Decentralized Vehicle Rental Management), a novel hybrid "Web 2.5" architecture that strategically partitions system responsibilities between on-chain and off-chain components to address these limitations. The system employs ERC-721 non-fungible tokens (NFTs) to represent vehicle ownership, enabling true digital ownership, composability with DeFi protocols, and access to secondary markets. An innovative event-sourcing pattern synchronizes on-chain state with a high-performance off-chain cache, achieving sub-second read latency while maintaining blockchain security guarantees for write operations. The architecture follows the principle of "Writes via Wallet, Reads via Cache," where high-trust, low-frequency transactions are executed on-chain through smart contracts, while high-frequency read operations are served from an optimized Node.js backend. This paper contributes a systematic design methodology for building performant decentralized applications, demonstrating that hybrid architectures can reconcile the conflicting demands of decentralization, user experience, and scalability in peer-to-peer rental marketplaces.**

***Index Terms*—Blockchain, Decentralized Applications, NFT, ERC-721, Event Sourcing, Web 2.5, Smart Contracts, Peer-to-Peer Marketplace**

I. Introduction

Decentralized applications (dApps) promise to create trustless, transparent, and peer-to-peer systems that can disintermediate traditional industries in finance, supply chain management, and marketplaces. Despite this potential, dApp adoption remains nascent, blocked by a fundamental "usability crisis" that is not merely aesthetic but deeply technical. This crisis is rooted in three core problems [7].

First is performance and latency. Operations on a blockchain are bound by network propagation, consensus mechanisms, and block generation times. This results in multi-second or even multi-minute latencies for interactions that users expect to be instantaneous. Second is the economic and cognitive load of gas fees. Every state-changing operation (a "write") has a computational cost, which is passed to the user as a gas fee. This creates direct economic friction and, more critically, a cognitive overhead where users must perform a constant, complex cost-benefit analysis for simple actions.

Third, these factors culminate in severe user-experience (UX) barriers. The dApp paradigm forces users to adopt unfamiliar and intimidating interaction models, such as managing non-custodial wallets, securely storing seed phrases, and signing opaque transactions. This contrasts sharply with the seamless, "single sign-on" experience of Web2 applications [9].

These problems form a causal chain. The foundational principle of decentralized consensus requires global state replication, which causes inherent latency and throughput limitations. This latency forces a "wait-to-confirm" interaction model, and the cost of this computation necessitates the gas mechanism. Together, these constraints make it functionally impossible to build a performant, "read-heavy" application (like a marketplace) in a naive, purely on-chain manner. Developers, forced by these constraints, create complex UX patterns, which ultimately results in the failure to achieve mainstream adoption [3].

The industry has primarily focused on scalability to solve these issues. The most prominent solutions are Layer-2 (L2) protocols, such as rollups and state channels. L2s are a critical innovation, effectively scaling transaction throughput by moving computation off-chain, thereby making "write" operations faster and cheaper. However, L2s are only a partial solution. They do not architecturally solve the problem of data querying. A dApp running on an L2 still lacks a high-performance interface for fetching and displaying thousands of data points—for example, all listings in a marketplace—without resorting to slow and costly Remote Procedure Call (RPC) node queries [12].

Decentralized indexing protocols, such as The Graph, have emerged to address this "read problem" directly. These systems, while philosophically aligned with the Web3 ethos, introduce their own significant overhead. Developers must build and deploy "subgraphs," and the systems are subject to their own token-economic models and potential performance trade-offs, such as query latency or data-freshness lag. This

leaves a clear architectural gap for developers seeking the performance of Web2 with the trust of Web3. This pragmatic, hybrid "Web 2.5" model is an emerging but under-formalized solution [10].

This paper formally defines and evaluates a "Web 2.5" hybrid architecture through the implementation of a Decentralized Vehicle Rental Management (dVRM) system, a peer-to-peer vehicle rental marketplace.

Our core contribution is the explicit partitioning of dApp responsibilities:

High-Trust "Write" Path: All operations involving value or state changes (e.g., asset minting, escrow) are handled as atomic, on-chain transactions, signed directly by the user's wallet. This preserves the core Web3 security model of self-custody and trustless execution.

Low-Trust "Read" Path: All operations involving data consumption (e.g., browsing, searching listings) are served by a high-performance, off-chain, centralized cache, providing a Web2-like, sub-second UX.

Specific contributions include:

- The formalization of the "Writes via Wallet, Reads via Cache" architectural pattern.
- A full-stack reference implementation (dVRM) demonstrating this pattern's viability for a read-heavy marketplace.
- A secure on-chain design leveraging ERC-721 for the Real-World Asset (RWA) tokenization of vehicles, which separates the asset-layer (`VehicleRegistry.sol`) from the logic-layer (`RentalAgreement.sol`) for enhanced security.

The remainder of this paper is organized as follows. Section 5 reviews related work in blockchain scalability and dApp architectures. Section 6 provides a detailed technical description of the proposed dVRM system methodology. Section 7 outlines the experimental setup for evaluating our system, and Section 8 presents the results and analysis. Section 9 discusses the implications and limitations of our findings. Finally, Section 10 concludes the paper.

FIX ME

II. Related Work

A. Blockchain Scalability Solutions

Research in blockchain scalability aims to address the transaction throughput limitations of Layer-1 (L1) chains. These solutions can be broadly categorized into L1 improvements (e.g., sharding) and Layer-2 (L2) protocols.

State channels, a type of L2 solution, enable off-chain interactions between a fixed set of participants after an initial on-chain transaction. While efficient for high-frequency, state-modifying interactions like a game, they are unsuitable for the dVRM use case, which requires a dynamic, globally-readable state (all available vehicles) accessible to an unknown number of potential renters [1].

Rollups (both Optimistic and Zero-Knowledge) are the dominant L2 solution for general-purpose scalability. They function by batching transactions off-chain and posting compressed data or validity proofs to the L1 chain. The dVRM architecture presented in this paper is orthogonal and complementary to L2s. The dVRM's smart contracts could be deployed on an L2 (e.g., Arbitrum) to reduce "write" gas costs. However, this deployment does not eliminate the need for our proposed caching layer. The L2 protocol itself still does not provide a high-performance query interface suitable for fetching thousands of listings, and thus the "read problem" remains [8].

B. Off-Chain Data and Querying Architectures

Solving the "read problem" requires a dedicated data-querying architecture. A common pattern is using off-chain storage like the InterPlanetary File System (IPFS) to store large data (e.g., images, documents) and placing only the IPFS hash on-chain. This is a pattern for storage, not querying. IPFS retrieval is known to have high latency and inconsistent availability, making it unsuitable as a primary database cache. The dVRM system employs this pattern only for static assets like the imageUrl [2].

The primary architectural alternative to our proposal is a decentralized indexing protocol, most notably The Graph. The Graph provides a decentralized network of "Indexers" that ingest blockchain data via "subgraphs" and serve it via a GraphQL API. This approach aims to create a fully decentralized, verifiable, and trustless query layer. However, this decentralization introduces significant implementation complexity, requires developers to learn a new stack, and subjects the system to token-economic incentives. Furthermore, it can suffer from performance issues, including query latency and indexing lag, which may not be suitable for real-time applications [10].

Our proposed centralized read cache is an emerging "best practice" that prioritizes performance and simplicity over the philosophical purity of full decentralization. This paper aims to formally analyze the trade-offs of this pragmatic pattern, as summarized in Table I.

C. Blockchain in Peer-to-Peer Marketplaces

A growing body of research has proposed the use of blockchain for peer-to-peer (P2P) marketplaces, particularly in real estate and vehicle rentals. These proposals rightly identify the benefits of blockchain for disintermediation, trust, and transparent record-keeping. Several papers have outlined systems for P2P car sharing. However, a common gap in this domain-specific literature is the proposal of naive, fully on-chain systems. Many studies focus on the smart contract logic for trust but fail to address the non-trivial performance and UX challenges that would render their systems unusable in practice. Our paper contributes a performant and usable architecture that makes these P2P marketplace concepts viable.

For example, the housing rental system proposed by Qi-Long, C. et al. correctly identifies the high cost of on-chain storage and utilizes IPFS to store large listing information, placing only the IPFS hash on-chain. This is a common pattern for data storage, but it does not solve the read latency

TABLE I
COMPARATIVE ANALYSIS OF DAPP DATA ACCESS ARCHITECTURES

Architecture	Read Latency (T_{read})	Data Freshness (Lag)	Decentralization (Reads)	Implementation Complexity
Pure L1/L2 Read	Very High	N/A (Direct)	High	Low
The Graph (Decentralized)	Medium	Medium (Polling)	High	High
dVRM Hybrid Cache (Ours)	**Very Low**	**Very Low (Event-Driven)**	**Low (Centralized)**	**Medium**

problem. In their model, a tenant must still query the on-chain "Listings Contract" to retrieve the hash before fetching the data from IPFS , an interaction model that is far too slow for a modern browsing experience. Our dVRM system, in contrast, uses a synchronized off-chain cache to serve all browsing data instantly, making the "Reads via Cache" model a core architectural principle, not just a storage solution [6].

Similarly, a P2P car-sharing system proposed by Pharande, R. et al. aims to remove centralized intermediaries using Ethereum smart contracts. Their architecture, however, implies that all user interactions—including ride requests and confirmations—are handled directly as on-chain smart contract authentications. This fully on-chain approach epitomizes the performance bottleneck that our dVRM system is explicitly designed to solve. The dVRM architecture partitions these actions, reserving on-chain transactions only for high-trust "write" operations, while serving the high-frequency "read" (browsing) path from the performant off-chain cache [5].

Other research has focused on different aspects of the problem, such as data representation. Kim, S. & Huh, J. propose an "Autochain platform" that uses an XML and XSL (stylesheet) format to handle unstructured data (like images) and automate smart contract creation from webforms. While this addresses data interoperability, it does not architecturally solve the high-frequency read-latency problem that is central to our dVRM's "Web 2.5" thesis [4].

Perhaps the most architecturally similar proposal is from Tseng, C. et al., which outlines a sophisticated hybrid, multi-layered system for housing rentals. Their system correctly identifies the need for trusted third-party verifiers (e.g., a Department of Land Administration) to certify assets and integrates a dedicated "Decentralized Identity (DID) Chain". However, their primary focus is on identity and verification. Their performance analysis identifies read latency as a bottleneck (e.g., in their GetAllOnlineListing function) and suggests caching as a future optimization. Our dVRM paper, by contrast, begins with the "Reads via Cache" pattern as its core architectural thesis, presenting a fully realized event-sourcing middleware as the solution to the performance problem, rather than identifying it as an area for future work. Our paper contributes a performant and usable architecture that makes these P2P marketplace concepts viable [11].

III. SYSTEM ARCHITECTURE: THE "WEB 2.5" HYBRID MODEL

The dVRM system is designed as a three-layer, decoupled stack:

- **On-Chain Layer (EVM):** The decentralized source of truth.
- **Off-Chain Middleware (Node.js):** The event-sourced read cache.
- **Off-Chain Frontend (React.js):** The user interface and interaction orchestrator.

The system's architecture (illustrated in Figure 1, not shown) is built upon the thesis: "Writes via Wallet, Reads via Cache." This partitioning is a deliberate "Web 2.5" compromise, positing that for a marketplace, the integrity of ownership and agreements (writes) must be decentralized, while the act of browsing (reads) is a low-trust operation that can be centralized for performance.

This "Web 2.5" model is an emerging and pragmatic approach that functions as a transitional architecture to drive mass adoption. It enables developers to leverage existing, performant Web2 infrastructure (like REST APIs and cached databases) for a seamless UX, thereby solving the onboarding and friction challenges that repel mainstream users. This model aligns with the philosophy of "Progressive Decentralization," where an application may begin with centralized components for performance and user-centricity, and then gradually decentralize specific components as the technology matures and the community grows.

Write Path (Decentralized, High-Trust): The user initiates a state-changing operation (e.g., registering a vehicle) via the frontend. The frontend uses `ethers.js` to request a signature from the user's wallet (e.g., MetaMask). The user signs and sends the transaction directly to the on-chain smart contract. The backend is explicitly bypassed for all high-trust operations.

Read Path (Centralized, Low-Trust): The user loads the website. The frontend makes a standard HTTP GET request to the off-chain middleware's REST API. The middleware instantly serves a JSON payload from its in-memory cache, providing a sub-second response.

IV. ON-CHAIN LAYER: THE DECENTRALIZED SOURCE OF TRUTH

The on-chain layer is the system's "bedrock" of trust, composed of two primary Solidity contracts.

A. VehicleRegistry.sol: The Asset Layer

This contract is an ERC-721 Non-Fungible Token (NFT) implementation, inheriting from OpenZeppelin's ERC721 and Ownable contracts. This is a key design decision, representing each vehicle as a unique NFT. This tokenizes the Real-World Asset (RWA), granting the owner true digital ownership, self-custody, and composability with the broader DeFi/NFT ecosystem (e.g., using the vehicle NFT as collateral).51

To enhance this composability, the contract stores all essential metadata (make, model, year, imageUrl, pricePerDay, status) within a `Vehicle` struct mapped to the `tokenId`. Storing this data on-chain, rather than just a hash to an off-chain metadata file, is more gas-intensive but makes the contract's state self-contained, allowing other smart contracts to read and interact with vehicle data trustlessly.

B. The "RWA Problem": Tokenizing Physical Assets

A critical theoretical challenge in this design is the "RWA Problem," which is the legal and practical disconnect between the digital token (the NFT) and the physical asset it represents (the vehicle). The NFT is a digital twin or claim on the asset, but it is not the asset itself. Legal scholars differentiate between the *corpus mysticum* (the intangible asset or right, such as intellectual property) and the *corpus mechanicum* (the physical object, like a car).

While the blockchain provides an immutable record of the NFT's ownership 54, this digital record does not automatically confer legally recognized ownership or possession of the physical car. An attacker could sell the NFT while retaining physical possession of the car, or sell the physical car while retaining the NFT, creating a significant legal dispute. This is often referred to as the "possession problem," as English law, for example, may not recognize an NFT as a possessable item. Solving this requires a robust "oracle" that bridges the legal and physical worlds—for example, by integrating with state-run digital title registries or IoT devices that can immobilize a vehicle if its corresponding NFT is not in the correct wallet. This paper acknowledges this significant limitation; our architecture focuses on providing a secure and performant management system for the digital claims, with the legal-physical bridging remaining a complex, external challenge.

C. RentalAgreement.sol: The Logic and Escrow Layer

This contract is a stateless logic engine. It does not maintain a list of agreements; rather, it manages state on the Registry contract. It interacts with the VehicleRegistry via a fixed `IVehicleRegistry` interface. Its core functions, `createRentalAgreement` and `completeRental`, manage the rental lifecycle.

When `createRentalAgreement` is called, it calculates the total rental cost (pricePerDay $\times$ duration) and requires the renter to attach that exact amount of ETH to the transaction. The contract holds this value in escrow. Upon successful completion, it transfers the funds to the owner; in a dispute, it could (with added logic) refund the renter.

D. The Inter-Contract Security Model

The lynchpin of the on-chain security model is the interaction between these two contracts. The VehicleRegistry contract contains a function `setVehicleStatus(tokenId, newStatus)` that updates a vehicle's state (e.g., from "Available" to "Rented"). This function is protected by a critical modifier:

```
modifier onlyRentalContract() {
 require(msg.sender == rentalAgreementAddress,
 "Invalid caller");
 _;
}
```

This modifier ensures that only the RentalAgreement contract can change a vehicle's rental status. This simple mechanism is what prevents owner fraud. A malicious vehicle owner cannot rent their vehicle, accept an escrow payment, and then manually call `setVehicleStatus` to set their vehicle back to "Available" to rent it to a second person. All state changes are governed exclusively by the agreed-upon rules in the logic engine.

V. Off-Chain Middleware: The Event-Sourced Read Cache

The middleware layer is a Node.js server using Express.js and Ethers.js. Its sole purpose is to build and maintain a high-performance, in-memory replica of the on-chain state. It achieves this using the Event Sourcing pattern, treating the blockchain's event log as the immutable, ordered source of truth. The process is twofold.

A. Justification for the Event Sourcing Pattern

A naive cache might attempt to replicate state by repeatedly querying the smart contracts directly (e.g., calling `getVehicle()` for every token). This approach is inefficient, scales poorly, and places a heavy load on the RPC node.

The Event Sourcing pattern provides a far more robust and scalable solution. Instead of storing the current state, we store the sequence of immutable events that led to that state. The blockchain's event log is, by its very nature, an append-only, immutable event store. Our middleware treats these logs (e.g., `VehicleRegistered`, `VehicleStatusUpdated`) as the canonical source of truth.

This pattern yields several key advantages:

- **Auditability:** The event log provides a complete, immutable history of every state change, which is invaluable for auditing, debugging, and analytics.
- **State Reconstruction:** We can reconstruct the system's state to any point in time simply by replaying the event log up to that point.
- **Performance and Decoupling (CQRS):** This pattern naturally implements Command Query Responsibility Segregation (CQRS). The "Command" side (writing) is handled by the on-chain smart contracts. The "Query" side (reading) is handled by our separate, optimized read model (the cache). This decoupling allows each side to be scaled independently and improves system performance.

B. Architectural Alternative: Comparison with State Channels

It is important to contrast our read-cache architecture with other prominent off-chain scaling solutions, namely State Channels. State channels allow a group of participants to conduct multiple transactions off-chain, submitting only an initial and final state to the main blockchain.

This model is highly effective for high-frequency, state-modifying interactions between a fixed set of participants (e.g., a poker game or a micropayment channel). However, state channels are fundamentally unsuitable for the dVRM use case for one critical reason: our application is a "read-heavy" marketplace that requires a globally-readable, dynamic state. A potential renter needs to browse all available vehicle listings, which are owned by thousands of different, unknown parties. State channels are inherently private and siloed between their participants; they do not provide a mechanism for an outside, un-channelled party to query this public-facing state. Our event-sourced cache, by contrast, is designed to solve this "many-reader, public-data" problem.

C. Phase 1: Historical Sync (syncPastEvents)

On startup, the server connects to an EVM RPC node. It uses the `ethers.js` function `registryContract.queryFilter()` to fetch all historical `VehicleRegistered` and `VehicleStatusUpdated` events, from the contract's deployment block (genesis) to the present. It iterates through these logs, replaying the entire history of the system to build a complete in-memory `Map<tokenId, Vehicle>` of the current state. This guarantees the cache is a perfect, complete reflection of the on-chain truth.

D. Phase 2: Real-Time Listening (listenToLiveEvents)

After the historical sync is complete, the server uses a persistent WebSocket connection via `registryContract.on(...)` to subscribe to live events. When any user's "write" transaction is confirmed on-chain and the contract emits an event, the backend listener 21 detects it in real-time (typically within 1–3 seconds of block confirmation). The listener's callback function then mutates the in-memory cache (e.g., adding a new vehicle or updating its status), ensuring the cache remains synchronized.

E. The Read Endpoint (GET /api/vehicles)

The server exposes a single, high-performance REST endpoint. A request to this endpoint performs no blockchain operations. It simply serializes the in-memory cache and returns it as a JSON array. This provides the sub-second, Web2-speed response required for a fluid user experience.

VI. Frontend Layer: Orchestrating the Hybrid Interaction

The frontend is a React.js application that orchestrates the two distinct interaction paths.

The Hybrid "Read" Flow (Web2 UX): A user loads the dVRM website. A `useEffect` hook in React triggers a standard `fetch('/api/vehicles')` request. The UI populates almost instantly with the JSON data. No wallet connection is required, no gas is paid, and no blockchain latency is experienced. This flow directly solves the UX barriers of complex onboarding and interaction friction.

The Hybrid "Write" Flow (Web3 Security): A vehicle owner fills out the "Register Vehicle" form. On submit, the application calls `const signer = provider.getSigner()` from `ethers.js`, which triggers the user's MetaMask wallet. The user is prompted to sign and send a transaction (paying the requisite gas) directly to the `VehicleRegistry.sol` contract's `registerVehicle()` function. The frontend bypasses the backend entirely for this security-critical operation. The application then waits for the transaction to be confirmed (e.g., `tx.wait()`).

The "Write-to-Read" Data Flow: The full loop demonstrates the system's elegance. (1) User A (Owner) completes the "Write" flow. (2) The `VehicleRegistered` event is emitted on-chain. (3) The Node.js backend's listener (Phase 2) detects this event. (4) The backend adds the new vehicle to its in-memory cache. (5) User B (Renter), browsing the site, will now receive this new vehicle in the JSON payload from the next `GET /api/vehicles` request. This entire loop completes in near-real-time.

VII. The Asymmetric Security Model

This architecture's trust is asymmetric. The "write" path is fully decentralized and trustless, while the "read" path is centralized and trusted for convenience only. A critical analysis must address the "malicious read-cache" threat model.

Assume the centralized Node.js server is compromised. A malicious operator could try to defraud a user by serving a fake JSON payload—for example, showing a vehicle's price as "1 ETH/day" when the true on-chain price is "0.1 ETH/day."

This attack is defeated by the architectural partitioning.

1) The renter sees the fraudulent "1 ETH" price and clicks "Rent."
2) The React frontend, following the "Write" path, bypasses the malicious backend.
3) It constructs a transaction to call `RentalAgreement.sol`, which reads the true price (0.1 ETH) from the on-chain source of truth, `VehicleRegistry.sol`.
4) The RentalAgreement contract calculates the escrow requirement as 0.1 ETH.
5) The user's MetaMask wallet prompts them with the true transaction: "You are about to send 0.1 ETH..."
6) The user will see the discrepancy between the (malicious) displayed price and the (correct) on-chain transaction price.

The attack fails, and no funds are lost. The architecture is therefore resilient to a malicious read-cache. The cache is purely for performance; trust is always re-established at the point of transaction.

A more subtle, and equally important, threat involves a compromise of the Web2 components themselves. For instance, a Cross-Site Scripting (XSS) attack could inject malicious JavaScript into the frontend. This script could attempt to deceive the user by altering the displayed price on the webpage.

However, the security model holds: the malicious script cannot forge the user's signature. When it constructs the fraudulent transaction (e.g., to drain the user's wallet), the

user's wallet (MetaMask) will present the actual details of that transaction. As long as the user scrutinizes the transaction details presented by the trusted wallet interface, the attack is mitigated. The wallet acts as the final, trusted checkpoint.

VIII. Discussion

A. Interpretation of Architectural Contribution

The "Web 2.5" hybrid model is a highly effective and pragmatic solution for read-heavy dApps. The architecture successfully isolates on-chain security from off-chain performance. The dVRM system is designed to achieve Web2-rivaling speeds and a user experience that solves the "dApp Usability Crisis", while the asymmetric security model preserves the decentralized trust and self-custody of the "write" path (Section 6.5). This approach breaks the causal chain where performance limitations historically led to an unusable product.

B. Limitations

This architecture is defined by its trade-offs, and it is crucial to state its limitations.

Centralization of Reads: The primary limitation is the centralized read-cache. If the dVRM backend server is offline, browsing the marketplace becomes impossible. This is the explicit trade-off made for performance, in contrast to a decentralized (but slower) solution like The Graph. This is an acceptable risk for a "low-trust" operation, but it remains a single point of failure for availability.

The "RWA Problem" (Legal and Practical): As discussed in Section 6.2.2, the second major limitation is that this system manages the digital ownership of the vehicle NFT, but not the legal, physical-world title. The NFT is a representation or claim on the physical asset, but there is no cryptographic link to the physical car keys or the state's title registry. Solving this "oracle problem" for physical assets is a complex legal and logistical challenge that is outside the scope of this paper's architectural contribution.

Event-Sourcing at Extreme Scale: While the in-memory map is highly performant for thousands of NFTs, this approach would not scale to tens of millions of assets. At that scale, the initial `syncPastEvents` process would be prohibitively slow, and the in-memory cache would exceed the capacity of a single server. A more robust implementation would replace the in-memory map with a persistent, horizontally-scalable database (e.g., Redis or PostgreSQL) that is still populated by the same event-sourcing logic.

IX. Threat Model, Limitations, and Future Work

While the hybrid architecture presented in this paper offers a practical bridge between decentralized trust guarantees and contemporary user-experience expectations, several open security and architectural concerns remain. Understanding these concerns is essential for evaluating the viability of similar Web 2.5 systems in production environments.

A. Cache Integrity and Data Freshness

Although the event-sourced cache is updated in near real-time, there is an unavoidable period between transaction confirmation and cache mutation. In high-frequency marketplaces, even a few seconds of inconsistency can create race conditions. Strategic attackers might exploit these transient windows to attempt front-running behavior offline. Future work should evaluate incremental Merkle proofs or lightweight, verifiable state commitments that allow clients to detect stale data.

B. Backend Compromise Scenarios

If the backend is compromised, it may selectively omit listings or artificially reorder them to promote favored assets. While the write path remains secure, the renter's discovery process becomes subtly biased. One potential mitigation is progressive decentralization of indexing: rather than relying on a single REST endpoint, multiple independently operated caches could be queried in parallel. Divergent responses would be statistically detectable.

C. Rental Dispute Resolution Oracles

The paper acknowledges that the "possession problem" is not just philosophical; it is operational. If a renter returns a car damaged, or refuses to return it at all, the NFT state alone is insufficient to resolve disputes. Future systems may integrate secure IoT modules, cryptographic ignition locks, or telematics sensors that attest to mileage and usage. The challenge lies in ensuring that such devices are tamper-resistant and legally recognized.

D. Regulatory Compliance and Digital Title Registries

The legitimacy of tokenized ownership ultimately depends on state recognition. Without statutory amendments to vehicle code, the NFT remains merely a record of intent, not possession. Legislative pilot programs could enable "dual-channel ownership," where digital title updates propagate into governmental systems. The fragility of this legal bridge is currently the most significant bottleneck for real-world NFT utility.

E. Zero-Knowledge Privacy Enhancements

The current dVRM design exposes all rental activity publicly. While transparency is useful for auditability, it may conflict with user privacy expectations. Integrating zero-knowledge circuits for selective disclosure could yield a privacy-preserving rental history. However, implementing ZK systems introduces complexity and cost, making them an ideal candidate for staged adoption in mature markets.

F. Interoperability With DeFi Primitives

Future versions of this architecture could allow vehicle-NFT holders to collateralize rental income streams, producing novel asset-backed lending primitives. However, this raises systemic risk questions: if a renter defaults on maintenance obligations, collateral valuations may change unpredictably. Asset-class volatility modeling would be required to avoid cascading liquidations in periods of market stress.

In summary, the hybrid Web 2.5 paradigm is not an endpoint; it is a transitional phase. The road ahead requires careful cryptographic engineering, regulatory negotiation, and human-centric dispute mediation mechanisms.

G. Economic Incentive Stability

A subtler concern lies in the incentive layer that motivates both renters and owners to behave honestly. Token-based deposits seem attractive at first, yet they introduce volatility risk unrelated to the underlying vehicle. If the asset backing the deposit crashes in value, deterrence collapses. Conversely, if it spikes, rational actors may refuse to participate altogether. Future research must evaluate hybrid collateral models that blend fiat-linked stable deposits with on-chain attestation of rental outcomes. Ignoring these economic ripples risks turning a functional marketplace into a speculative battleground.

H. Latency as Market Power

Performance improvements are often praised without reservation, yet history shows that speed tends to centralize. Operators who run exceptionally fast caches quietly gain influential gatekeeper power; they can reorder results, prioritize profitable listings, or subtly degrade visibility for competitors. A marketplace that claims neutrality cannot depend solely on goodwill. Periodic audits, signed result manifests, and multi-party cache federation could help dilute the gravitational pull of low-latency monopolies. Without such safeguards, the architecture risks drifting toward soft centralization despite its noble intent.

I. User Experience Versus Cryptographic Transparency

A recurring tension in decentralized design is the conflict between convenience and verifiability. Users want clean interfaces and rapid responses, yet deep inspection of proofs and Merkle commitments rarely fits into that narrative. Hiding cryptographic details within glossy UI components may lull users into false confidence. Future iterations of this architecture should investigate layered interfaces that adjust transparency based on user profile. Veteran users may inspect intermediate proofs, while newcomers can rely on summarized attestations. Ignoring this split risks creating a two-tier culture of blind trust.

J. Operational Burden and Lifecycle Maintenance

Even if the system works flawlessly on launch day, real vehicles age: odometers change, components break, and accidents occur. Mapping this messy, physical lifecycle onto a crisp digital twin is nontrivial. Periodic inspection proofs, signed maintenance records, and insurer participation could help align on-chain representations with off-chain realities. Without these controls, the NFT drifts away from the asset it claims to represent, becoming an outdated certificate rather than a living record. The integrity of the marketplace depends on shrinking that drift, not pretending it is negligible.

X. Conclusion

Decentralized applications have long promised to revolutionize peer-to-peer interaction, but their potential has been crippled by fundamental performance and usability barriers that prevent mainstream adoption. A purely on-chain architecture is too slow and costly for a modern, data-rich application, and the resulting user experience is unacceptable.

This paper has formalized, implemented, and theoretically analyzed a pragmatic "Web 2.5" hybrid architecture, "Writes via Wallet, Reads via Cache," using a P2P vehicle rental dApp (dVRM) as a case study. Our analysis indicates that by intelligently partitioning dApp responsibilities—handling high-trust "writes" on-chain and high-frequency "reads" via an event-sourced off-chain cache—our system can achieve sub-second, Web2-like read performance and a superior user experience.

Crucially, we have shown this performance is achieved without compromising the core Web3 security guarantees of decentralized trust and user self-custody, as the architecture is resilient to a malicious read-cache. This pragmatic hybrid pattern offers a validated, high-performance blueprint for developers, serving as a vital bridge to build the next generation of scalable, user-centric decentralized applications.

Acknowledgment

The author extends sincere gratitude to the mentors and peers who offered thoughtful critique during early architectural iterations. Their skepticism sharpened the system's threat model and prevented several naive assumptions from making their way into production. Appreciation is also due to the broader open-source community, whose tooling and documentation formed the quiet scaffolding beneath every component described here.

References

[1] What are layer 2 solutions in blockchain? `https://www.geeksforgeeks.org /ethical-hacking /what-are-layer-2-solutions-in-blockchain/`, September 2024. Accessed: 2025-11-3.

[2] Vero Estrada-Galiñanes, Ahmad ElRouby, and Léo Marc-André Theytaz. Efficient data management for IPFS dapps. April 2024.

[3] D. Khan, L. T. Jung, and M. A. Hashmani. Systematic literature review of challenges in blockchain scalability. *Applied Sciences*, 11(20):9372, October 2021.

[4] Seong-Kyu Kim and Jun-Ho Huh. Autochain platform: expert automatic algorithm blockchain technology for house rental dapp image application model. *EURASIP J. Image Video Process.*, 2020(1), December 2020.

[5] Rutuja Pharande, Shubhangi Gunjal, Abhishek Mahale, and Prof Aparna Patil. Peer-to-peer car-sharing system using blockchain technology. *Int. J. Res. Appl. Sci. Eng. Technol.*, 11(5):4762–4771, May 2023.

[6] Chen Qi-Long, Ye Rong-Hua, and Lin Fei-Long. A blockchain-based housing rental system. In *Proceedings of the International Conference on Advances in Computer Technology, Information Science and Communications*. SCITEPRESS - Science and Technology Publications, 2019.

[7] J. Saldivar, E. Martínez, D. Rozas, M. C. Valiente, and S. Hassan. Blockchain (not) for everyone: Assessing the user experience of blockchain-based applications. *SSRN Electronic Journal*, 2022.

[8] Cosimo Sguanci, Roberto Spatafora, and Andrea Mario Vergani. Layer 2 blockchain scaling: a survey. July 2021.

[9] N. Sonule. User experience (ux) in web3 and dapps: Challenges and opportunities. *International Journal of Scientific Harmonization of Digital Horizons (IJSHODH)*, 1(1):19–21, July 2025.

[10] Y. Tal. Introducing the graph. Medium, July 2018. Available: `https://medium.com/graphprotocol/introducing-the-graph-4a281b28203e`. [Accessed: 03-Nov-2025].

[11] Ching-Hsi Tseng, Yu-Heng Hsieh, Yen-Yu Chang, and Shyan-Ming Yuan. Towards a trustworthy rental market: A blockchain-based housing system architecture. *Electronics (Basel)*, 14(15):3121, August 2025.

[12] J. Xie, F. R. Yu, T. Huang, R. Xie, J. Liu, and Y. Liu. A survey on the scalability of blockchain systems. *IEEE Network*, 33(5):166–173, September 2019.